

Experiment 6: Scalable Read APIs with Caching & Optimization

This document explains the advanced backend concepts implemented in the Student Task Manager project, directly following the requirements of Experiment 6. It also provides **real-world production examples** to justify why these techniques are essential in industry-standard applications.

1. Pagination & Sorting

Why it's needed: When dealing with thousands (or millions) of database records, fetching all of them at once will crash the backend due to memory exhaustion (OutOfMemory exceptions), spike database load, and slow down frontend rendering.

How it's implemented:

- In `TaskRepository.java`, we inherit from `JpaRepository`, which automatically supports pagination.
- In `StudentTaskManagerApplication.java`, the `GET /api/tasks` endpoint accepts a `Pageable` object. By passing `?page=0&size=5&sort=id,desc` in the URL, the API returns a `Page<Task>` containing only 5 records and metadata like `totalPages`.

 **Production Example (E-Commerce):** Imagine Amazon loading the "Laptops" category. If Amazon tried to load all 50,000 laptops at once, the database would lock up and the user's browser would freeze. Instead, they use Pagination and Sorting (`size=24`, `sort=price_low_to_high`) to only load exactly what the user is currently viewing on page 1.

2. N+1 Problem and JOIN FETCH

Why it's needed: The "N+1 Problem" is a common ORM (Object-Relational Mapping) pitfall. If we fetch a list of 50 Tasks (1 query), and then loop through them to fetch their Comments, Hibernate fires 50 additional queries (N queries). Firing 51 queries instead of 1 severely degrades database performance and clogs the network.

How it's implemented:

- We added a `Comment` entity that has a `@ManyToOne` relationship with `Task`.
- In `TaskRepository.java`, we wrote a custom JPQL query: `@Query("SELECT DISTINCT t FROM Task t LEFT JOIN FETCH t.comments")`.
- This `JOIN FETCH` forces Hibernate to pull both Tasks and Comments in a **single** efficient database round-trip using an SQL `LEFT OUTER JOIN`.

 **Production Example (Social Media):** Think of Twitter/X. When loading a feed of 100 Tweets, you also need to load the Author details (Name, Profile Pic) for each tweet. If Twitter suffered from the N+1 problem, loading a feed would take 101 database queries. Using `JOIN FETCH`, they retrieve the tweets and the associated authors in exactly 1 query, ensuring the feed loads instantly.

3. Caching with Ehcache / Spring Cache

Why it's needed: If an API endpoint is called frequently but its underlying data rarely changes, repeatedly querying the database is a massive waste of CPU, memory, and database connection limits.

How it's implemented:

- We added `@EnableCaching` to the main application class.
- In `TaskService.java`, we annotated `getAllTasksWithComments()` with `@Cacheable("tasks")`. The first time this is called, it hits the database. Subsequent calls return the cached data from application memory in less than a millisecond.
- We added `@CacheEvict(value = "tasks", allEntries = true)` to the `addTask`, `deleteTask`, and `toggleTask` methods so the cache is invalidated and refreshed whenever the underlying data changes.

 **Production Example (Content Delivery):** Consider Netflix's movie catalog. The list of movies in "Trending Now" doesn't change every single second, but millions of users request it simultaneously. Netflix caches this API response in memory (e.g., Redis or Memcached). So instead of hitting the core database millions of times per second, 99.9% of users get a blazing-fast response directly from the cache.

4. Native SQL Queries

Why it's needed: While JPQL (Java Persistence Query Language) and ORMs are great for standard CRUD operations, they sometimes generate inefficient SQL for highly complex operations. Sometimes, you need the raw power of database-specific SQL to tune performance.

How it's implemented:

- In `TaskRepository.java`, we used `@Query(value = "SELECT * FROM tasks ORDER BY id DESC LIMIT 5", nativeQuery = true)`.
- The `nativeQuery = true` flag tells Spring Data JPA to skip JPQL parsing and execute the exact SQL string provided against the underlying H2 database.

 **Production Example (Financial Tech / Analytics):** In a banking app (like Stripe or PayPal), generating a monthly financial summary might involve highly complex SQL features like `Window Functions`, `Common Table Expressions (CTEs)`, or `PIVOT` tables. JPQL doesn't support these advanced database-specific features natively. Engineers must use Native SQL to let the database engine calculate the heavy analytics natively rather than pulling millions of rows into Java memory to calculate them manually.

5. Transition to H2 Database

Why it's needed: Using in-memory lists (`ArrayList`) limits applications to basic memory operations. Real applications require ACID properties (Atomicity, Consistency, Isolation, Durability), transactional safety, indexing, and persistent storage.

How it's implemented:

- We imported `spring-boot-starter-data-jpa` and `h2` database dependencies.
 - We mapped the `Task` class to a database table using the `@Entity` annotation.
 - Spring Boot automatically initializes the database schema and handles JDBC connections seamlessly.
-

6. Architecture & Folder Structure

To implement these enterprise-level patterns cleanly, the monolithic backend was refactored into a standardized layered architecture (Controller → Service → Repository → Entity).

Backend (/backend)

```
backend/
├─ pom.xml                # Maven configuration including Spring Data JPA, H2, and
Cache starters
├─ src/main/
│   └─ resources/
│       └─ application.properties # Configurations for the H2 Database and
Hibernate
│   └─ java/com/example/backend/
│       └─ StudentTaskManagerApplication.java # Application entry point and REST
API Controllers
│   └─ entity/
│       ├── Task.java      # JPA Entity defining the 'tasks' table schema
│       └─ Comment.java    # JPA Entity defining the 'comments' table (for N+1
demonstration)
│   └─ repository/
│       └─ TaskRepository.java # Data Access Layer containing JPQL and Native
SQL queries
│   └─ service/
│       └─ TaskService.java  # Business Logic Layer managing transactions and
@Cacheable logic
```

Frontend (/frontend)

```
frontend/
├─ package.json           # Node.js dependencies
├─ src/
│   ├── App.jsx            # React component containing state management and
paginated Fetch API calls
│   ├── App.css            # Premium CSS styling (glassmorphism, micro-animations,
gradients)
│   └─ index.css           # Global CSS resets
```